

SIGIL True Instant Finality

A BFT Finality Gadget Layered on DagKnight

Design proposal v0 · 2026-08-25

Rocky (engineering agent) · for operator review

IN PLAIN WORDS — What this document is

Right now a SIGIL block only becomes “safe to trust” after 512 more blocks are built on top of it (roughly the chain’s confirmation wait) — and even then it is a *probabilistic* guarantee, not a mathematical one: the code’s own comment admits no finite wait is safe against an unbounded adversary. **True instant finality** means a block becomes permanently irreversible the instant a quorum of trusted signers vote for it — zero chance of ever being reversed, by construction, not by odds. This is a real multi-week engineering project. This document is a concrete design for it, grounded in how production systems (Ethereum, Polkadot, Sui) actually solve the same problem, and in what this codebase already has today. **No code has been written.** This is the design to react to before any is.

Contents

1	The core recommendation	1
1.1	Why not copy Sui’s Mysticeti instead	1
2	Validator set	2
2.1	The codebase already answers “how many validators are enough”	2
2.2	Three options, weighed against where SIGIL actually is	3
3	Message format & wire protocol	3
4	Safety and liveness, in plain terms	3
5	Phased implementation plan	4
6	Honest risk assessment	4

1 The core recommendation

Split block *production* from block *finality* into two independent layers.

- **Layer 1 — production, unchanged.** DagKnight keeps mining, ordering, and coloring blocks exactly as it does today. Nothing about how a block gets made changes.
- **Layer 2 — finality, new.** A small, known, fixed set of signers (*validators*) periodically look at what DagKnight has already produced and cryptographically *vote*: “I agree this specific point in the order is correct.” Once enough of them (a *quorum*) sign the *same* thing, that point is **finalized** — locked forever. No future block, no longer-looking competing history, nothing, can ever overturn it again.

IN PLAIN WORDS — Why two layers instead of one

Today, SIGIL uses one mechanism (DagKnight’s block-count wait) to do both jobs at once: decide the order, *and* decide when to stop worrying about it changing. The two-layer split is how every modern chain that wants real finality solves this: **Ethereum** runs Casper FFG (voting) on top of GHOST (production, DagKnight’s closest academic cousin); **Polkadot** runs GRANDPA (voting) on top of BABE (production). Polkadot’s own docs put it exactly right: “By using separate protocols for block production and finality, Polkadot can achieve rapid block creation and strong guarantees of finality . . . GRANDPA guarantees that once a block is finalized, it can never be reverted.” Production never has to change, slow down, or become more centralized to get real finality — the voting layer sits on top, watching, not steering.

1.1 Why not copy Sui’s Mysticeti instead

Mysticeti (live on Sui mainnet since mid-2024) is the most relevant *recent* DAG-native BFT system, and deserves the direct comparison the operator asked for. It is **leaderless and uncertified**: validators build the DAG by referencing each other’s blocks directly; a block is considered decided once enough later blocks reference it — there are no separate signed vote messages at all. Per the technical writeup: “Mysticeti advances the DAG by receiving $n - f$ messages, not by certifying $n - f$ signatures,” committing in three network round-trips with sub-second latency at real scale on Sui mainnet.

The catch: in Mysticeti, **the DAG is the vote**. Production and finality are not separable layers — the validator committee *is* the set of block producers, and their act of referencing each other’s blocks *is* the voting. Retrofitting this onto SIGIL would mean replacing DagKnight’s open, PoW-gated production model with a closed, committee-authored DAG — ending permissionless mining entirely. That is a legitimate, much larger future direction; it is not the “add finality on top of what exists” project requested here. A cross-check against the field’s own survey literature (a 2024 SoK on DAG-based consensus) found no published design that hybridizes GHOSTDAG-style probabilistic fork-choice with a Mysticeti-style uncertified DAG-BFT layer — this proposal’s split-layer shape is closer kin to GRANDPA/Casper FFG, deliberately.

What *is* borrowed from the newer literature: the **quorum-certificate mechanics**. HotStuff-2 (2023) showed that a classic three-phase BFT vote can be safely simplified to two phases — “vote, then a quorum certificate *is* the commit” — with no loss of safety, because DagKnight (not the finality gadget) already handles proposing/ordering. Its production deployment, Jolteon/AptosBFT, reports sub-second finality under normal conditions using exactly this simplification. This proposal uses HotStuff-2’s two-phase quorum rule, GRANDPA/Casper FFG’s layering shape, and treats Mysticeti’s commit-rule idea (fast implicit commit via later references) as a candidate future optimization, not the base design.

2 Validator set

2.1 The codebase already answers “how many validators are enough”

`sigil-braidpool::committee` already implements the standard Byzantine-quorum formula:

$$\begin{aligned} f_{\max}(n) &= \left\lfloor \frac{n-1}{3} \right\rfloor && \text{(tolerable Byzantine/offline validators)} \\ \text{quorum}(n) &= n - f_{\max}(n) && \text{(signatures needed to finalize)} \\ \text{bft_active}(n) &\iff n \geq 4 && \text{(minimum for BFT to mean anything)} \end{aligned}$$

	n (validators)	f (tolerable)	quorum needed	real BFT?
	1	0	1	No — “trust the one signer,” not BFT
2 (Epsilon + happysrv, today)		0	2 (both)	No — and either going offline halts finality
	4	1	3	Yes — the code’s own documented floor
	7	2	5	Yes, more headroom
	10	3	7	Yes, more headroom

HONEST RISK

SIGIL's real, current, live producer count is **2** (Epsilon and `happysrv`) — below the codebase's own minimum for a meaningful Byzantine-fault-tolerance claim. “Just let today's producers vote” is not viable as-is: a 2-of-2 quorum halts finality the instant either node has a routine restart, and offers no real safety margin against a single compromised key. Reaching $n \geq 4$ needs at least two more independent machines under operator control.

2.2 Three options, weighed against where SIGIL actually is

Option	What it means	Fit for SIGIL today
A. Fixed, operator-configured	Operator directly picks $n=4-5$ validator keys, config-pinned, changed only by a deliberate coordinated action	Recommended. Simplest; honest about SIGIL's current operator-run reality; matches the existing <code>SIGIL_TRUSTED_PRODUCER_ID_HEX</code> “every node agrees out-of-band” precedent already in this codebase
B. Stake-weighted / open registration	Anyone stakes or registers a key; voting power proportional	Not yet — needs a real sybil defense first, and this codebase already has an unresolved, related risk (fork-choice weighted by raw block count, not real work) that opening validator registration would extend into the safety layer, not just production
C. Delegated / elected committee	Token-holders elect a small validator committee	Reasonable <i>future</i> step once there is a real, widely-held token to elect from — not now

Recommendation: Option A, $n=4$ or 5, Ed25519 keys (reusing the exact signing scheme already live for producer authentication), pinned in a config every node loads at startup.

3 Message format & wire protocol

Reuse Ed25519 for every vote (cheap, already wired in `producer_signing.rs`); reuse the existing periodic hybrid SQIsign5+Ed25519 checkpoint scheme (already built, every 128 blocks) to additionally post-quantum-attest only the deepest finality anchors, not every vote — SQIsign5 costs $\sim 1.16s$ per signature by the code's own measurement, too slow for per-vote use.

```
struct FinalityVote {
    height: u64,
    spine_block_hash: BlockHash, // DagKnight's own selected spine at that height
    order_hash: [u8; 32],        // Braid's existing frozen_acc order commitment
    validator_id: ValidatorId,
    signature: SignatureBytes,
}
```

A *certificate* is simply the set of votes agreeing on the same (height, spine_block_hash, order_hash) once that set reaches quorum size. Propagation reuses the existing gossip-topic convention (`/sigil/g0/blocks`, `/sigil/g0/peer-heights`, ...) with one new topic, `/sigil/g0/finality-votes`.

HONEST RISK

Before any Phase 2 code is written: confirm which gossipsub subscription list the *live* `sigil-node` binary actually builds from. `sigil-net`'s own topic-constant list has a documented dead-code trap — a same-named constant lives in a different crate (`flux_p2p`) and is the one actually wired in; this exact class of mistake already shipped once (v7.1.44) and quietly did nothing. Verify before wiring, don't assume.

4 Safety and liveness, in plain terms

The one-sentence proof. If two *different* things both somehow got finalized, each would need its own quorum of $2f+1$ signatures out of $n = 3f+1$ total. Two such quorums must overlap in at least $(2f+1) + (2f+1) - (3f+1) = f+1$ validators — meaning at least one *honest* validator (since at most f are dishonest) would have had to sign both conflicting things. An honest validator never does that by construction. This is the standard Byzantine quorum-intersection argument underlying GRANDPA, Casper FFG, and HotStuff-2 alike — not new math, but the exact reason “irreversible” is a real claim here and not marketing language, *as long as the $\leq f$ assumption holds*.

What happens	Consequence
$\leq f$ validators go offline / partition	Finality <i>pauses</i> for that checkpoint; production keeps going untouched; catches up once reconnected. A liveness hiccup, never a safety break.
$> f$ validators go offline at once	Finality halts entirely until enough return. Nothing unsafe happens, but nothing new finalizes — a direct, real cost of a small n , and not hypothetical given this fleet’s own operational history (Delta down a month; Gamma/Beta permanently gone).
A validator equivocates ($\leq f$ dishonest)	Quorum-intersection holds; no conflicting certificate can form. Both signed votes are visible on the wire — cheap, strong evidence for a future slashing/eviction mechanism.
$> f$ actually dishonest (assumption broken)	No BFT design of any kind can prevent this — mitigation is entirely about keeping n small enough that the operator directly knows every key (Option A) and making the set genuinely hard to expand quietly.

HONEST RISK

This gadget makes whatever DagKnight *already selected* permanent — it does not audit or correct that selection. If the existing count-based (not difficulty-weighted) blue-score metric can be gamed to bias which block becomes the spine, the finality gadget will lock in a gamed result exactly as confidently as an honest one, *forever*. Today a bad selection is only temporary — outrunnable by enough honest work. After finality, it would not be. Treat fixing blue-score to weigh real work, not raw block count, as a prerequisite to seriously evaluate before Phase 3, not a nice-to-have.

5 Phased implementation plan

Phase	Ships	Consensus effect	Size	Risk if wrong
1	Vote/certificate types, Ed25519 sign/verify, quorum-intersection property tests	None — pure library code	Days	Low — caught by tests
2	P2P gossip topic, local vote tallying, “would-be finalized height” logged alongside today’s estimate; <code>Braid::insert()</code> untouched	None — purely observational, same posture as the existing frontier-debug diagnostic	1–2 wks	Low — worst case is bad logs
3	Braid actually gated on real certificates, behind an opt-in flag, isolated test network only	This is the consequential phase	Weeks	Chain-safety bug if wrong
4	Live activation on sigil-g0, height-gated, after Phase 3 survives adversarial soak testing	Full — real funds depend on correctness	Smaller if Phase 3 was thorough	Irreversible by design if wrong

6 Honest risk assessment

A bug in ordinary block production is a rollback-and-fix situation. A bug in a *finality* gadget is categorically worse, because the entire point of the feature is the promise that nothing can be undone — getting it wrong either permanently locks in a wrong outcome, or forces breaking the very promise the feature exists to make, which is worse than never shipping it.

Before Phase 3 ever touches the live chain, this proposal insists on:

1. A dedicated multi-node test network, $n \geq 4$ validators on *genuinely separate* machines, run for real wall-clock days, not a CI pass.
2. Explicit adversarial scenarios: kill a validator mid-vote repeatedly; force one to equivocate deliberately; partition the validator set so neither side can reach quorum; replay the exact reorder-depth scenario that already forced the 64→512 `final_depth` bump once before.
3. A genuine go/no-go conversation with the operator before Phase 4 — finality is a bigger trust commitment than the existing “adding a second producer deserves its own explicit go-ahead” norm already practiced in this codebase, and deserves at least the same bar.
4. The count-vs-work blue-score gap resolved or consciously, explicitly accepted first.
5. A written-down-in-advance answer for “what do we do if Phase 3/4 finds a safety bug after something is already finalized live” — decided before activation, not improvised after.

Decision points needing the operator

1. Go/no-go on Option A (fixed, small, operator-controlled validator set) versus waiting for a real stake/token distribution to justify Option B/C.
2. Where do the 2+ additional validator machines come from — new independent boxes/VPS instances under operator control.
3. Fix the blue-score count-vs-work gap before Phase 3, or explicitly accept it and proceed in parallel.
4. Confirm whether producer-signing (`SIGIL_PRODUCER_SIGNING_SEED_HEX`) is already active on live Epsilon — a config file suggests it might be, contradicting a code comment that says it is dormant everywhere.